GENERATED 2026-04-01

CTF隐写从0到1教程

这是一套面向 CTF 初学者的隐写教程，目标不是把所有方法讲成学术论文，而是让你形成一套能落地的做题流程：

1. 先判断这是不是隐写题
2. 再确定是哪一类隐写
3. 最后用最小成本把有效线索挖出来

如果你刚接触 CTF，这套资料建议按顺序阅读。如果你已经做过一些题，可以直接从 `03-常用工具与排查流程.md` 开始，把它当赛时 checklist 使用。

目录

- `01-基础认知.md`：隐写是什么、常见分类、解题总流程
- `02-图片隐写与文件拼接.md`：图片题最常见的切入方式，包含 JPEG+ZIP 和 OpenSSL 密文实战
- `03-常用工具与排查流程.md`：赛时最有用的命令、工具和决策树
- `04-更复杂的隐写方法.md`：频域隐写、自适应隐写、音视频隐写、网络隐写、AI watermark
- `05-练习路线与做题清单.md`：从入门到进阶的练习路径与复盘模板
- `06-音频与视频隐写.md`：音频、视频、频谱、拆帧与时间轴隐写实战
- `07-实战题单与训练计划.md`：按阶段安排的练习题型、训练方法和复盘模板
- `08-常见题型速查表.md`：按题型、特征、首选工具和坑点整理的一页速查
- `practice/`：可直接动手练习的样本题包，按题型分目录，附提示、解法与统一答案索引
- `scripts/Extract-AppendedZip.ps1`：提取 JPEG/PNG/GIF 等文件尾部追加的 ZIP
- `scripts/Decrypt-OpenSslSalted.ps1`：解 OpenSSL `Salted__` 格式的 AES-CBC Base64 密文
- `scripts/Generate-PracticeSamples.ps1`：一键生成练习区中的样本附件
- `scripts/media_builder/`：为高级媒体题生成 GIF/WebM/WAV/二维码/PCAP 等样本所需的本地 Node 构建脚本
- `CTF隐写从0到1-完整版.md`：整合版讲义源稿
- `CTF隐写从0到1-完整版.docx`：整合版 Word 讲义
- `CTF隐写从0到1-完整版.pdf`：整合版 PDF 讲义

适合谁

- 刚开始接触 CTF Misc/隐写

- 知道一些工具名，但不会组织排查流程
- 能看懂命令输出，但不知道下一步该做什么
- 想把“做过一道题”升级成“知道一类题怎么打”

你会学到什么

- 如何快速识别隐写题的入口
- 如何判断题目是文件拼接、元数据、像素域、频域还是密码学组合题
- 如何在 Windows PowerShell 环境下直接排查
- 如何把“提取载荷”和“解密载荷”拆成两个独立步骤
- 如何处理音频频谱、视频抽帧和时间轴线索
- 如何建立自己的赛时排障思路
- 如何用现成样本附件做从 0 到 1 的实操练习

实操练习现状

当前 `practice/` 已包含基础到进阶的样本题，覆盖：

- 文件尾拼接
- OpenSSL 二层密文
- PNG alpha
- 低对比度隐藏字
- Base64 + Gzip 编码链
- 音频摩斯
- JPG 口令载荷题
- PNG 调色板题
- GIF 拆帧题
- 视频抽帧题

继续扩展后，`practice/` 现已包含 17 道样本题，额外补充：

- PNG LSB / zsteg 思路题
- XMP 元数据题
- 音频频谱写字题
- 元数据 + 密文的双层题
- 图片 + 元数据 + 密文 + 假 flag 三层题
- GIF + 二维码 + Base64 组合题
- DNS 流量隐写入门题

建议阅读顺序

1. `01-基础认知.md`
2. `02-图片隐写与文件拼接.md`
3. `03-常用工具与排查流程.md`
4. `06-音频与视频隐写.md`
5. `05-练习路线与做题清单.md`
6. `08-常见题型速查表.md`
7. `07-实战题单与训练计划.md`
8. `04-更复杂的隐写方法.md`

学习方式建议

- 不要只看正文，边看边自己敲命令
- 每学完一节，至少做一道对应类型的小题
- 每道题都保留一份复盘：入口、工具、关键证据、最终 flag
- 不要迷信工具输出，先理解文件结构再谈一键梭哈

一句话总纲

隐写题真正考的不是“你会多少工具”，而是“你是否能把文件格式、异常特征、密码学线索和脚本验证串成一条稳定流程”。

---

01 基础认知

1. 什么是隐写

隐写的核心不是“加密”，而是“隐藏存在感”。

- 加密关注的是：别人看到数据，也读不懂
- 隐写关注的是：别人最好连“这里有数据”都没意识到

在 CTF 里，这两者经常叠加出现：

- 第一层是隐写：把内容藏进图片、音频、文档、流量、压缩包、文件尾部
- 第二层是加密：即使提取出载荷，还要再解码、解密、爆破口令

所以做隐写题时，不要默认“提出来就结束了”。很多题真正的坑在第二层。

2. CTF 中常见的隐写类型

2.1 文件结构类

这类题不一定改图片像素，而是利用文件格式的容错性或解析差异。

典型形式：

- `JPEG + ZIP` 文件拼接
- PNG 里插入异常 chunk
- GIF 后面挂额外数据
- 一个文件同时可被当成两种格式解析

特点：

- 最适合先做十六进制、文件头、文件尾检查
- 人眼看图通常看不出异常
- 但二进制结构里会留下强证据

2.2 元数据类

信息藏在：

- EXIF
- XMP
- 注释段
- ICC profile
- 文档属性

特点：

- 提取快
- 隐蔽性一般
- 新手最容易漏，因为总把注意力放在图像内容本身

2.3 像素域隐写

典型是 LSB。

做法是把数据塞进：

- RGB 通道最低位
- Alpha 通道
- 调色板索引

特点：

- 教学题很多
- 容量大
- 如果嵌入得粗糙，统计特征明显

2.4 频域隐写

常见于 JPEG。

它不直接改像素，而是改压缩过程中使用的 DCT 系数。常见思路：

- JSteg
- F5
- OutGuess

特点：

- 比简单 LSB 隐蔽
- 更像正规隐写
- 新手不容易直接看出来

2.5 多媒体隐写

包括：

- 音频隐写
- 视频隐写
- 帧提取
- 频谱/相位/回声隐写

特点：

- 线索可能分散在多个时间片
- 容量大
- 很多题会把二维码、文本或压缩包拆在多帧里

2.6 网络隐写

把信息藏进：

- DNS 查询
- HTTP 头
- 包长度

- 时间间隔
- TLS 元数据

这类题更偏取证和流量分析，不是传统图片隐写，但思维方式一致：先找异常，再还原载荷。

3. 做题的标准流程

建议你形成下面这套固定流程。比赛时紧张，流程越稳定越重要。

第一步：识别文件类型

先不要急着“跑 steghide”。

要先确认：

- 文件扩展名和真实格式是否一致
- 是单文件还是文件夹
- 是否有明显的伴随文件
- 文件大小是否可疑

你应该优先问自己：

- 这真的是一张纯图片吗
- 它会不会只是个容器
- 它后面会不会拼了别的东西

第二步：检查文件结构和元数据

优先排查：

- 文件头
- 文件尾
- 结束标记后是否有多余数据
- EXIF/XMP/Comment
- 是否出现 `PK`，`Rar!`，`7z`，`SQLite`，`PDF`，`flag`，`Salted__` 等关键字

第三步：提取所有“看起来不像图片本体”的东西

比如：

- 文件尾追加的 ZIP
- PNG 异常 chunk
- Base64 字符串

- 看起来像十六进制、二维码、压缩流的片段

这一步的目标不是立刻得 flag，而是把“隐写层”和“解密层”分离。

第四步：判断是不是还叠了编码或加密

常见信号：

- `U2FsdGVkX1`：OpenSSL `Salted__`
- `H4sIA`：Gzip Base64
- `eJ`：zlib 压缩的 Base64
- 一长串高熵数据：可能是加密结果
- 长度刚好对齐：可能是块加密

第五步：再回头检查图像/音频/视频内容本身

如果文件结构没问题，再去查：

- 通道分离
- LSB
- 色彩反相
- 对比度拉伸
- 频谱图
- 帧拆分

顺序不要反。很多人一上来就盯像素，效率很低。

4. CTF 隐写里最常见的组合拳

题目经常不是单点，而是两层或三层：

- 图片尾部拼 ZIP + ZIP 里有加密文本
- EXIF 给口令 + steghide 真正藏数据
- GIF 帧里有二维码 + 二维码内容再 Base64
- 音频频谱出文本 + 文本是维吉尼亚或栅栏密码

所以你要习惯问：

- 这个线索是终点，还是只是下一层的钥匙

5. 初学者最容易犯的错

错误 1：只会按工具名乱试

这会导致：

- 浪费时间
- 看不懂输出
- 对失败没有解释

正确做法：

- 先根据格式和异常选工具

错误 2：提取到东西就以为结束

比如提取到：

- Base64
- 十六进制
- 压缩流
- 假 flag

这通常只是中间层。

错误 3：忽略文件尾

文件尾追加是 CTF 非常高频的套路。因为：

- 好做题
- 好出题
- 学习价值高

错误 4：忘记记录证据链

正确的解题复盘应该能回答：

- 为什么你怀疑这里有隐写
- 你用什么证据确认
- 你怎么提取
- 你怎么解密
- 中间有哪些误导项

6. 你现在就该记住的赛时口诀

先看类型，再看结构；
先提载荷，再谈解密；

先抓证据，再跑工具；
一个结果，不要默认就是终点。

---

02 图片隐写与文件拼接

这一章只讲 CTF 里最常见、最值得优先检查的一类：图片文件相关隐写。

1. 先理解为什么“图片后面拼 ZIP”成立

以 JPEG 为例：

- JPEG 以 `FFD8` 开头
- JPEG 以 `FFD9` 结束

对大多数图片查看器来说，只要读到 `FFD9`，图片就结束了。
如果有人在 `FFD9` 之后再追加一段 ZIP 数据：

- 图片依然能正常显示
- 但整个文件从二进制角度看已经变成“图片 + 额外载荷”

这就是最经典的文件拼接隐写。

2. JPEG 里最值得先看的结构

常见标记：

- `FFD8`：SOI，开始
- `FFE0`：APP0，常见 JFIF
- `FFE1`：APP1，常见 EXIF
- `FFDA`：SOS，扫描数据开始
- `FFD9`：EOI，结束

赛时实用结论：

- 重点看 `FFD9` 后有没有额外字节
- 重点看 APP 段里有没有 EXIF/XMP/Comment

3. JPEG+ZIP 的实战排查

3.1 思路

排查顺序：

1. 找到 JPEG 结束位置
2. 计算结束后还有没有尾随数据
3. 看尾随数据是不是 `PK 03 04`
4. 如果是，就按 ZIP 解

3.2 PowerShell 示例

```
$path = '题目.jpg'
$bytes = [System.IO.File]::ReadAllBytes($path)

$lastEOI = -1
for ($i = 0; $i -lt $bytes.Length - 1; $i++) {
    if ($bytes[$i] -eq 0xFF -and $bytes[$i + 1] -eq 0xD9) {
        $lastEOI = $i
    }
}

"EOI 位置: $lastEOI"
"尾随字节数: " + ($bytes.Length - ($lastEOI + 2))
```

如果尾随字节数大于 0，再看尾巴：

```
$tail = $bytes[($lastEOI + 2)..($bytes.Length - 1)]
($tail[0..15] | ForEach-Object { '{0:X2}' -f $_ }) -join ' '
```

如果开头是：

```
50 4B 03 04
```

那就是 ZIP。

3.3 直接提取 ZIP 尾巴

```
$zipBytes = $bytes[($lastEOI + 2)..($bytes.Length - 1)]
[System.IO.File]::WriteAllBytes('hidden.zip', $zipBytes)
```

然后你就可以正常解压它。

4. 这类题的典型第二层：载荷再加密

很多题不会把真 flag 直接塞进 ZIP，而是放一层诱饵。

例如：

```
flag{this_flag_is_a_lie}
U2FsdGVkX1/zdL/aAU73J9nRW9GWY2nOOHefWO0JiUnr3kMEcXUtXNS0bOeDTGOM
```

第一行看起来像 flag，但题目明确暗示它是假的。
第二行才是关键，它是一个 Base64 字符串。

5. 如何识别 OpenSSL `Salted__` 密文

看到下面这种 Base64 前缀时要立刻警觉：

```
U2FsdGVkX1
```

它 Base64 解码后通常以：

```
53 61 6C 74 65 64 5F 5F
```

也就是：

```
Salted__
```

这说明它很可能是 OpenSSL `enc` 生成的加密结果。

常见参数组合：

- `AES-128-CBC`
- `AES-192-CBC`
- `AES-256-CBC`
- `-salt`
- 旧题中常见 `MD5` 做 EVP_BytesToKey 派生

6. 典型解题套路

假设你已经从图片尾部提取出 `flag.txt`，内容如上。

这时你的正确思路应该是：

1. 不要把第一行假 flag 当终点

2. 先判断第二行是什么格式
3. 发现它是 `Salted__`
4. 再思考第一行是不是口令
5. 用第一行作为 password 去解第二行

这就是非常典型的：

- 隐写层：文件拼接
- 解密层：OpenSSL 密文

7. 对应的 OpenSSL 命令

如果系统里有 `openssl`，可以直接试：

```
$cipher = 'U2FsdGVkX1/zdL/aAU73J9nRW9GWY2nOOHefWO0JiUnr3kMEcXUtXNS0bOeDTGOM'
$cipher | openssl enc -aes-256-cbc -d -a -salt -md md5 -pass pass:"flag{this_flag_is_a_lie}"
```

如果参数猜对，就会输出明文。

8. 不装 OpenSSL 也能解

本教程附带了：

- `scripts/Decrypt-OpenSslSalted.ps1`

用途：

- 输入 Base64 密文
- 输入口令
- 按 OpenSSL 常见的 `AES-CBC + MD5 EVP_BytesToKey` 方式尝试解密

9. 图片内容本身还要不要查

要，但优先级在文件结构之后。

合理顺序是：

1. 先看结构
2. 再提尾巴
3. 再查元数据
4. 最后才看通道、反相、LSB、对比度拉伸

因为在很多入门到中级题里，文件结构异常的命中率非常高。

10. 图片内容隐写的基础入口

如果结构和元数据都没异常，再考虑这些方向：

- 通道分离：R/G/B
- 反相、拉高对比度
- 查看 Alpha
- LSB bit plane
- PNG 调色板顺序
- JPEG DCT 隐写

10.1 通道分离

目的：

- 看文字是否只藏在某一个颜色通道里
- 看是否有局部涂层或低透明度覆盖

10.2 LSB

最低位隐写最常见的特征：

- 正常图像基本看不出来
- 但 bit plane 展开后会出现纹理异常、图案、二维码、文字边缘

10.3 JPEG 频域

如果文件是 JPEG，不要执着于 LSB。
因为 JPEG 是有损压缩，很多像素级方法并不稳定。此时更该考虑：

- JSteg
- OutGuess
- F5

11. 这一类题的赛时最小流程

拿到一张图后，先按下面顺序走：

1. 看文件头尾是否正常
2. 看 `FFD9` 后有没有数据
3. 看尾巴是否是压缩包或其他文件头

4. 查 EXIF/XMP/Comment
5. 查是否出现 `flag`，`Salted__`，`PK`，`Base64`
6. 没收获再做通道、LSB、频域

12. 这一章的核心结论

很多所谓“图片隐写题”，本质上根本不是像素隐写，而是：

- 图片作为伪装外壳
- 载荷藏在文件结构外围
- 真正难点落在后续编码或加密

所以你做题时一定要分清：

- 你是在找“东西藏哪了”
- 还是在解“拿出来以后它是什么”

---

03 常用工具与排查流程

这一章最适合比赛时直接翻。

1. 赛时排查总流程

拿到题目文件后，优先级建议如下：

1. 确认真实文件类型
2. 检查元数据和文件结构
3. 查文件尾、可打印字符串、可疑魔数
4. 提取所有非本体载荷
5. 判断是否叠加编码/加密
6. 结构没异常，再看像素域/频域/音视频层

2. 最常用工具清单

2.1 通用取证

- `file`：识别真实格式
- `strings`：提取可打印字符串

- `xxd` / 十六进制查看器：看文件头尾和魔数
- `binwalk`：扫压缩包、固件、内嵌格式
- `foremost`：按文件头批量雕刻

2.2 图片相关

- `exiftool`：元数据
- `stegsolve`：图层、通道、反相、位平面
- `zsteg`：PNG/BMP LSB 检测
- `steghide`：经典隐写工具，常见于 JPG/WAV
- `outguess`：JPEG 隐写
- `pngcheck`：PNG chunk 检查
- `identify` / `magick identify`：图片属性

2.3 音视频相关

- `ffmpeg`：拆帧、抽音轨、转码
- `ffprobe`：查看流信息
- Audacity：频谱、相位观察

2.4 密码学配套

- `openssl`
- CyberChef
- 自己写的小脚本

3. Windows PowerShell 常用命令

3.1 查文件大小和基本信息

```
Get-ChildItem -LiteralPath '.\题目文件'
```

3.2 读文件头前 64 字节

```
$bytes = [System.IO.File]::ReadAllBytes('.\题目.jpg')
($bytes[0..63] | ForEach-Object { '{0:X2}' -f $_ }) -join ' '
```

3.3 找关键字

```
$latin1 = [System.Text.Encoding]::GetEncoding('ISO-8859-1').GetString($bytes)
@('PK','Salted__','flag','Exif','JFIF','PNG','GIF89a') | ForEach-Object {
    "$_ => " + $latin1.IndexOf($_)
}
```

3.4 查 JPEG 结束后是否有尾巴

```
$lastEOI = -1
for ($i = 0; $i -lt $bytes.Length - 1; $i++) {
    if ($bytes[$i] -eq 0xFF -and $bytes[$i + 1] -eq 0xD9) {
        $lastEOI = $i
    }
}

"Trailing bytes: " + ($bytes.Length - ($lastEOI + 2))
```

3.5 直接把尾巴存下来

```
$tail = $bytes[($lastEOI + 2)..($bytes.Length - 1)]
[System.IO.File]::WriteAllBytes('.\tail.bin', $tail)
```

4. 图片题的决策树

情况 A：文件尾有额外数据

优先怀疑：

- ZIP/RAR/7z 拼接
- 二进制载荷
- 自定义密文

动作：

- 提取尾巴
- 识别魔数
- 尝试按压缩包打开

情况 B：有 EXIF/XMP/Comment

优先怀疑：

- 口令藏在元数据

- 直接藏了 Base64
- 给出下一层提示

动作：

- 完整导出元数据
- 看拍摄软件、作者、注释、用户字段

情况 C：结构正常，但 PNG/BMP

优先怀疑：

- LSB
- Alpha
- 调色板

动作：

- `zsteg`
- `stegsolve`
- 通道分离

情况 D：结构正常，但 JPEG

优先怀疑：

- steghide
- outguess
- F5 / JSteg 一类频域隐写

动作：

- 先试口令线索
- 再试 JPEG 专项工具

情况 E：音频/视频

优先怀疑：

- 频谱文本
- 拆帧二维码
- 低音轨/高音轨分层
- 元数据或字幕轨

动作：

- `ffprobe`
- `ffmpeg -i`
- 抽帧
- 频谱观察

5. `strings` 类思维非常重要

即使没有 `strings` 命令，你也应该有同样的意识：

- 文件里有没有可打印文本
- 有没有 Base64
- 有没有压缩包标记
- 有没有盐值、算法头、JSON、XML、URL

这一步极其高性价比。

6. 比赛时最该先试哪些口令

如果工具提示需要 password，不要乱爆破。先从题面和文件里找：

- 文件名
- 题目名
- 作者名
- 假 flag
- EXIF 注释
- 图片上可见文本
- 附件名
- 同目录其他文件名

CTF 里口令通常不是完全随机，而是“能被你推出来”的。

7. 什么时候该自己写脚本

满足下面任意一个条件，就别纠结 GUI 了，直接写脚本：

- 需要重复试多组口令
- 需要批量扫多个文件
- 需要解析简单固定格式
- 工具输出太多，你只想提取其中一个字段
- 你已经确认原理，只差自动化

8. 常见误判

误判 1：看到假 flag 就停

正确做法：

- 看它是不是诱饵
- 看是否还有更高熵的东西

误判 2：所有图片题都先跑 LSB

正确做法：

- 先按格式分类
- JPEG 先结构和频域
- PNG/BMP 再优先像素域

误判 3：把工具报错当“没有隐写”

很多报错只是：

- 口令不对
- 格式不匹配
- 文件损坏
- 工具不支持该方法

9. 一份赛时 checklist

下面这份可以直接照着过一遍：

```
[ ] 文件扩展名和真实格式一致吗
[ ] 文件头尾正常吗
[ ] 结束标记后有多余数据吗
[ ] 有 EXIF/XMP/Comment 吗
[ ] 有明显字符串、Base64、Salted__、PK 吗
[ ] 能提取出额外压缩包或文本吗
[ ] 结果是终点还是下一层线索
[ ] 是否需要口令，口令线索来自哪里
[ ] PNG/BMP 是否查过 LSB/Alpha/Palette
[ ] JPEG 是否考虑过 steghide/outguess/F5
[ ] 音视频是否拆帧/看频谱
```

10. 这一章的核心目标

把工具变成流程的一部分，而不是碰运气的按钮。

---

04 更复杂的隐写方法

这一章不是让你赛时手搓论文级算法，而是帮你建立“更难题大概长什么样”的判断力。

1. 为什么有些隐写比文件拼接难很多

因为好的隐写不会留下明显的结构破绽。

文件拼接的问题在于：

- 文件尾异常明显
- 魔数容易识别
- 提取载荷很直接

更复杂的方法会尽量做到：

- 文件结构完全正常
- 元数据也正常
- 改动分散
- 不影响肉眼观感
- 不容易被简单统计抓出来

2. LSB 的升级版

2.1 基础 LSB

最简单的方法是把每个像素某一通道的最低位改成消息位。

问题：

- 统计异常比较明显
- 被裁剪、压缩后容易损坏

2.2 LSB Matching

不是直接硬改某一位，而是根据目标位决定像素值加一或减一。

优点：

- 统计痕迹比朴素 LSB 小

2.3 伪随机嵌入

不是按顺序写入，而是根据密钥控制写入位置。

优点：

- 不知道密钥的人不容易定位载荷

2.4 只在纹理区嵌入

例如只在：

- 草地
- 头发
- 云层
- 噪声高的区域

优点：

- 更不容易被肉眼发现
- 更不容易在简单差分图里暴露

3. JPEG 频域隐写

3.1 为什么 JPEG 适合频域

JPEG 在压缩时会把图像分块后做 DCT 变换，所以数据天然经过频域表示。

更隐蔽的做法就是：

- 不碰像素
- 改压缩系数

3.2 常见方法

- JSteg
- F5
- OutGuess

3.3 赛时信号

如果：

- 文件结构完全正常
- 又是 JPEG
- 元数据没线索
- 图片本体也没可见异常

就应该把注意力转到频域工具。

4. 自适应隐写

这类方法不追求“每个像素都能藏”，而是追求“藏得不容易被发现”。

核心思想：

- 在平滑区少改
- 在复杂区多改
- 尽量让整体统计特征看起来正常

常见名字：

- HUGO
- WOW
- S-UNIWARD

CTF 中不一定会要求你完整复现这些算法，但可能借用它们的思路。

5. Polyglot 与格式解析差异

有些文件能同时被两个程序识别成不同格式。

例如：

- 图片里嵌文档
- PDF 里嵌脚本
- 压缩包和图片共存

难点不在算法，而在：

- 你是否意识到“一个文件可能有两种身份”

6. PNG 里的隐藏空间

PNG 比 JPEG 更适合玩结构题，因为它是 chunk-based 格式。

常见入口：

- `IHDR`
- `PLTE`
- `IDAT`
- `IEND`
- 自定义 chunk
- `IEND` 之后尾随数据

你要特别注意：

- chunk 长度是否异常
- chunk 类型是否陌生
- `IEND` 之后是否还有内容

7. 音频隐写

常见方式：

- LSB
- 相位编码
- 回声隐藏
- 频谱可视化
- 左右声道差分

CTF 常见做法：

- 把文本写进频谱图
- 多段音频拼起来才有意义
- 把二维码分布在不同时刻

8. 视频隐写

常见方式：

- 帧内藏图
- 抽帧后拼接
- 字幕轨
- 音轨里藏东西
- 运动向量或编码残差

做题经验：

- 一定记得音频轨和视频轨要分开分析

- 一定记得按时间轴抽帧

9. 网络隐写

更偏实战和取证：

- DNS 子域名编码
- HTTP header 顺序
- 包大小模式
- 时间间隔编码

它最像隐写本质：不是把明文藏进一个文件，而是把信息藏进“行为模式”。

10. AI watermark / 机器学习隐写

这一类可以分两种：

10.1 平台私有 watermark

比如某些生成式平台会加肉眼不可见标记。

特点：

- 需要专用检测器
- 通用工具通常解不出内容

10.2 神经网络隐写

把消息编码进图像中，让图像整体更像自然分布。

特点：

- 更隐蔽
- 但往往依赖配套解码器

11. 哪种最难

通常不是“算法名字最长”的最难，而是满足下面这些条件的最难：

- 文件结构完全正常
- 没有明显字符串
- 修改量很小
- 嵌入位置由密钥控制

- 提取后还叠加加密

所以高难题常见结构是：

- 正常 JPEG
- 频域隐写
- 口令来自题面
- 提取结果还是一层加密文本

12. 面对复杂隐写的正确心态

不要想着“一把梭”。

正确做法：

1. 先排除低成本方向
2. 再确认文件格式和可能的载体层
3. 最后再上专项工具或脚本

复杂题的关键从来不是会多少工具，而是你是否能稳定地缩小搜索空间。

---

05 练习路线与做题清单

1. 从 0 到 1 的练习顺序

建议按下面顺序练，不要一上来就追高难度。

第一阶段：文件结构和元数据

目标：

- 看懂文件头尾
- 会查元数据
- 会判断是否有追加内容

推荐练习：

- JPEG/PNG 尾部拼接压缩包
- EXIF 里藏 flag 或口令
- 文本文件伪装成图片扩展名

第二阶段：基础图片隐写

目标：

- 理解 RGB/Alpha/LSB
- 会做通道分离
- 会看 bit plane

推荐练习：

- PNG LSB
- Alpha 通道藏二维码
- 调色板顺序题

第三阶段：JPEG 专项

目标：

- 知道什么时候该怀疑频域
- 会尝试 `steghide`、`outguess`
- 会从题面或元数据找密码

推荐练习：

- 需要口令的 steghide
- 图片中可见提示词作为密码
- EXIF 给口令的 JPEG 隐写

第四阶段：音视频

目标：

- 会抽帧
- 会看频谱
- 会拆音轨和视频轨

推荐练习：

- 频谱出文字
- 抽帧出二维码
- 视频多帧拼图

第五阶段：综合题

目标：

- 能识别多层结构
- 会写简单脚本自动化
- 会做复盘

推荐练习：

- 图片拼接 + 密文
- 音频频谱 + Base64
- GIF 帧 + 二次加密

2. 一道题该怎么复盘

每做完一道题，建议按这个模板记：

```
题目名：
文件类型：
第一异常点：
确认异常的证据：
提取出的中间载荷：
中间载荷的类型：
是否还有第二层编码/加密：
最终 flag：
这题最关键的判断：
如果重做一次，最快路径是什么：
```

3. 新手最值得反复练的 10 个动作

1. 看文件头尾
2. 查结束标记后是否有尾随数据
3. 查 EXIF/XMP/Comment
4. 在字节流里搜 `PK`
5. 在字节流里搜 `Salted__`
6. 把尾巴切出来单独分析
7. 判断可疑字符串是不是 Base64
8. 判断提取结果是不是压缩流或加密结果
9. 看 PNG 的 Alpha 和 bit plane
10. 给自己写一个 20 行以内的小提取脚本

4. 赛时时间分配建议

如果你是单人做题，建议这么控时：

- 前 5 分钟：文件类型、元数据、文件尾、关键字
- 再 10 分钟：提取可疑载荷、判断编码/加密
- 再 10 到 20 分钟：像素域、频域或音视频专项
- 超时后：立刻整理证据，切换方向，不要死磕

5. 如何判断该不该爆破口令

不要无脑字典。

先问：

- 题面有没有提示
- 文件名是不是提示
- 作者名是不是提示
- 目录名是不是提示
- 假 flag 会不会就是密码
- 元数据里有没有词

如果没有任何来源，盲爆通常性价比很低。

6. 一份赛时速查模板

```
[输入]
文件路径：
文件大小：
真实格式：

[结构]
是否有尾随数据：
是否有异常 chunk / marker：
是否有元数据：

[载荷]
是否提取到文本：
是否提取到压缩包：
是否提取到 Base64/Hex：

[第二层]
是否像密文：
是否需要口令：
口令线索来源：

[专项]
是否查过 LSB/通道/Alpha：
是否查过 JPEG 专项工具：
```

```
是否查过音频/视频轨：

[结果]
最终 flag：
剩余疑点：
```

7. 进阶建议

当你把上述内容练顺以后，再去补：

- JPEG 压缩和 DCT 基础
- PNG chunk 结构
- WAV/MP3 基础
- ffmpeg 基础
- 简单密码学识别
- 自己写 PowerShell/Python 自动化脚本

8. 最后的建议

CTF 隐写真正的门槛不在工具，而在习惯。

如果你能稳定做到：

- 先看结构
- 先找异常
- 先分离载荷
- 再谈解密

那你已经不是“碰运气做题”，而是在建立自己的方法论。

---

06 音频与视频隐写

这一章的目标不是把多媒体处理讲成数字信号处理课程，而是让你遇到音频和视频隐写题时，知道应该先看什么、后看什么。

1. 为什么音视频隐写适合出题

和单张图片相比，音视频的冗余更高，藏数据的方法也更多：

- 时间轴可以切分
- 左右声道可以分离
- 频谱可以画图
- 视频可以拆成大量帧
- 一道题可以天然分成多层线索

这意味着：

- 信息可以藏在某一个时刻
- 也可以分散到整段时间里
- 还可以把“提取”和“拼接”设计成两步题

2. 音频题先看什么

拿到音频文件，建议先做这四步：

1. 看基础信息：时长、采样率、声道数、编码格式
2. 分析波形和频谱
3. 检查元数据和可打印字符串
4. 再考虑 LSB、相位、回声、分段拼接

2.1 基础信息

先搞清楚：

- 是 `wav`、`mp3`、`flac` 还是别的格式
- 是单声道还是双声道
- 时长是否过长或过短
- 码率和采样率是否异常

你真正想知道的是：

- 这题是“声音内容题”
- 还是“文件结构题”

3. 音频题最常见的几类

3.1 频谱出文字

这是 CTF 里最经典的音频题之一。

做法通常是：

- 把文本、二维码、图案画在频谱上
- 肉耳未必明显异常
- 但在频谱视图里一眼能看到

赛时动作：

- 用 Audacity 或其他工具打开频谱视图
- 调整窗口大小、FFT size、对数坐标
- 看高频和低频有没有文字或轮廓

3.2 左右声道差分

信息可能藏在：

- 左声道单独有内容
- 右声道单独有内容
- 两个声道相减后出现文本或节奏

所以音频题不要只听“混合后的总效果”，要分开看左右声道。

3.3 LSB 音频隐写

常见于未压缩音频，尤其是 `wav`。

特点：

- 原理和图片 LSB 类似
- 常需要专门工具或脚本提取

3.4 反向播放/变速/静音段

很多音频题并不复杂，只是把线索藏在：

- 反放
- 慢放
- 提速
- 静音段
- 某个非常短的片段

这类题比算法题更像“观察题”，但比赛里非常高频。

4. 音频题的实战流程

推荐顺序：

1. `ffprobe` 看流信息
2. 查元数据
3. 看波形
4. 看频谱
5. 拆左右声道
6. 导出若干关键时间片
7. 最后再怀疑高级隐写

5. 视频题先看什么

视频题的关键是分层分析。

你至少要把它拆成三层看：

- 视频画面
- 音频轨
- 容器和字幕/附件流

很多新手只盯着画面，这是错误的。
正确思路是先问：

- 线索在画面里吗
- 在音轨里吗
- 在字幕或元数据里吗
- 还是在帧序列变化里

6. 视频题最常见的几类

6.1 拆帧题

典型特征：

- 肉眼看视频正常
- 但逐帧拆开后能看到：
- 文字
- 二维码
- 多帧拼图
- 定时闪烁内容

赛时动作：

- 抽帧

- 低频采样浏览
- 再重点看可疑时间点

6.2 每隔 N 帧取一张

有些题目会把信息拆到：

- 奇数帧
- 偶数帧
- 每 5 帧
- 每秒一帧

这类题如果只正常播放，很容易错过。

6.3 视频里其实藏的是音频题

很多视频题真正的核心在线索音轨里：

- 频谱图
- 摩斯码
- DTMF
- 左右声道差异

所以视频题也经常需要先把音频抽出来单独分析。

6.4 字幕轨/附件流

容器格式里可能还带：

- 字幕
- 附件
- 章节
- 注释

如果你只看播放画面，会完全漏掉这层。

7. `ffmpeg/ffprobe` 最值得会的几类操作

7.1 看流信息

```
ffprobe 题目.mp4
```

重点看：

- video/audio/subtitle 是否都有
- 时长
- 编码器
- 分辨率
- 比特率

7.2 抽音轨

```
ffmpeg -i 题目.mp4 -vn audio.wav
```

7.3 拆帧

```
ffmpeg -i 题目.mp4 frames/frame_%05d.png
```

7.4 每秒抽一帧

```
ffmpeg -i 题目.mp4 -vf fps=1 frames/frame_%05d.png
```

8. 做音视频题时的常见信号

信号 1：时长明显异常

比如：

- 文件很大但时长很短
- 文件很小但时长很长

这通常说明：

- 码率异常
- 有隐藏流
- 数据结构可疑

信号 2：波形很奇怪

例如：

- 某段几乎静音但其实有信息
- 左右声道差异巨大
- 波形整齐得像编码数据

信号 3：画面有周期性闪烁

这通常意味着：

- 有帧间藏图
- 有交替帧
- 有时间维隐写

9. 音视频题的误区

误区 1：只靠耳朵听

很多题的关键信息不是“听出来”，而是“看出来”：

- 频谱
- 波形
- 声道差分

误区 2：只看正常播放效果

很多视频题必须拆帧才能看见。

误区 3：不会分层

正确拆法应该是：

- 容器层
- 流层
- 时间层
- 内容层

10. 这章的最小赛时 checklist

```
[ ] 看过流信息了吗
[ ] 有几条音轨/字幕轨
[ ] 把音频单独抽出来了吗
[ ] 看过频谱了吗
[ ] 左右声道分开看了吗
[ ] 抽帧看过了吗
[ ] 有没有按时间间隔采样
[ ] 有没有可疑闪烁或隐藏二维码
[ ] 是否检查过元数据和字符串
```

11. 这一章的核心结论

音视频隐写并不一定比图片更“高级”，但它更依赖分层思维。
你一旦养成“容器、流、时间轴、内容”四层拆解习惯，很多原本看起来复杂的题会立刻清晰很多。

---

07 实战题单与训练计划

这一章不直接给线上题目链接，而是给你一套可执行的训练框架。这样即使你换题库、换平台，训练方式也不会失效。

1. 训练目标

把“我会一点隐写”变成“我能稳定打穿一类题”。

具体拆成四个能力：

1. 识别载体类型
2. 快速定位异常
3. 分离隐写层和解密层
4. 形成可复用的脚本和复盘模板

2. 四周训练计划

第 1 周：文件结构与元数据

目标：

- 看到文件先查结构
- 会看头尾
- 会找魔数
- 会看 EXIF/XMP/Comment

每天建议：

- 1 道图片/压缩包拼接题
- 1 次独立复盘

本周题型：

- JPEG+ZIP

- PNG 尾随数据
- 假扩展名文件
- EXIF 藏密码

第 2 周：图片像素域

目标：

- 通道分离熟练
- 知道何时查 Alpha
- 知道怎么读 bit plane

本周题型：

- PNG LSB
- Alpha 二维码
- 调色板顺序
- 低对比度隐藏字

第 3 周：JPEG 与组合题

目标：

- 知道 JPEG 题不该无脑查 LSB
- 会先查结构、再查频域、再查密码
- 会识别 `Salted__`、Base64、压缩流

本周题型：

- steghide 口令题
- outguess 口令题
- 图片拼接 + 密文
- 假 flag + 真密文

第 4 周：音视频与综合题

目标：

- 会抽帧
- 会看频谱
- 会拆音轨
- 会建立完整复盘

本周题型：

- 频谱出文字
- 视频闪帧二维码
- 音轨 + 字幕双层线索
- 文件结构 + 音视频内容混合题

3. 每类题至少做到什么程度

文件结构题

最低标准：

- 10 分钟内定位追加数据
- 能独立提取尾部载荷

元数据题

最低标准：

- 能完整导出 metadata
- 能从 metadata 判断密码线索

图片像素题

最低标准：

- 能区分 RGB/Alpha/LSB 的入口
- 能从 bit plane 里读出明显文字或二维码

JPEG 题

最低标准：

- 不会先盲目跑 PNG/BMP 的套路
- 会把 steghide/outguess 和密码线索结合起来

音视频题

最低标准：

- 会抽帧、抽音频、看频谱
- 知道先分层再判断

4. 推荐训练方式

4.1 按题型刷，不按平台刷

不要今天随便做一道、明天再随便做一道。
更高效的方法是：

- 连续做 5 到 10 道同类题

这样你会很快看出模式：

- 什么线索最常见
- 哪些工具真的有用
- 哪些操作只是浪费时间

4.2 每类题都留一个“模板解法”

例如：

- 图片尾部拼接：固定检查脚本
- OpenSSL `Salted__`：固定解密脚本
- 视频抽帧：固定 `ffmpeg` 命令
- 频谱分析：固定观察顺序

比赛时你就不用临时重新想。

4.3 每周至少自己手工造 1 道小题

这一步非常重要。

自己出题时你会真正理解：

- 为什么这个点能藏
- 为什么那个点容易被发现
- 怎样叠一层误导最自然

建议先从下面几种开始自己做：

- `JPEG + ZIP`
- `EXIF 给密码 + 密文`
- `PNG Alpha 藏二维码`
- `音频频谱写字`

5. 做题记录模板

建议每道题都记录下面这份信息：

```
题目名：
题目来源：
附件类型：
初始怀疑方向：
第一个异常证据：
使用工具：
中间产物：
中间产物判定依据：
最终 flag：
本题最浪费时间的一步：
下次如何更快：
```

6. 如何衡量自己有没有进步

不是看“我今天做出几道”，而是看：

- 你是否更快地锁定正确方向
- 你是否更少地乱试工具
- 你是否能写出自己的复用脚本
- 你是否能向别人讲清楚证据链

7. 一份可执行的周复盘模板

```
本周练了哪些题型：
本周最熟的一类题：
本周最薄弱的一类题：
本周新增的两个工具或命令：
本周自己写了什么脚本：
本周遇到的一个假线索：
下周重点补哪一类：
```

8. 训练中的优先级建议

如果时间有限，优先顺序建议：

1. 文件结构题
2. 元数据题
3. PNG/BMP 图片像素题
4. JPEG 口令题
5. 音视频题
6. 高级自适应/学术型隐写

原因很简单：

- 前四类在 CTF 里命中率最高
- 也是最容易形成稳定方法论的部分

9. 最后给你的实战建议

真正把人拉开差距的，不是记住多少算法名，而是这三件事：

1. 先找异常，再找工具
2. 先拆层次，再做解密
3. 先写复盘，再谈熟练

只要你能持续按照这个节奏训练，隐写题会从“玄学”变成“流程题”。

---

08 常见题型速查表

这一页专门给赛时快速翻。
目标不是解释原理，而是帮你在最短时间内决定第一步应该做什么。

1. 图片类

题型	入口特征	首选工具/动作	常见坑点
JPEG 尾部拼接	文件能正常看图，但体积偏大；`FFD9` 后有数据；能搜到 `PK`	查 `FFD9` 后尾随字节、提取尾巴、识别 ZIP/RAR/7z	很多人一上来先跑 LSB，浪费时间
PNG 尾部拼接	`IEND` 后还有内容；体积异常	查 `IEND` 后数据、binwalk、十六进制查看	只看图片内容，忽略 chunk 结束后尾巴
EXIF/XMP 藏信息	图片正常，题面无明显入口	`exiftool`、查注释/作者/软件字段	元数据不一定直接给 flag，更可能给密码
低对比度隐藏字	画面局部发灰、过亮、过暗	提高对比度、反相、拉曲线	容易误把压缩噪声当文字
RGB 通道藏字	某一颜色区域看着不自然	通道分离、单独看 R/G/B	只看合成图时常看不出来

题型	入口特征	首选工具/动作	常见坑点
Alpha 通道藏图	PNG 边缘透明异常，背景切换后露出线索	查看 Alpha、换背景、导出透明层	很多人忘了 PNG 还有 Alpha
LSB 位平面	图片结构正常，PNG/BMP 更常见	`zsteg`、stegsolve、bit plane	JPEG 上无脑查 LSB 通常命中率低
调色板题	PNG/GIF 索引色图片	查看 palette 顺序、调色板导出	只看像素，不看索引
steghide 口令题	JPG/WAV，题面暗示 password	`steghide info/extract`，优先从题面/文件名找口令	盲目爆破性价比很低
outguess/F5/JSteg	JPEG 结构正常，元数据也正常	outguess、JPEG 专项工具	把所有 JPEG 都当 PNG 做像素排查

2. 压缩包与容器类

题型	入口特征	首选工具/动作	常见坑点
假扩展名	扩展名像图片，文件头不像图片	查文件头、真实格式识别	只信扩展名
压缩包套压缩包	解出一层后还是高熵数据或新容器	binwalk、strings、继续识别魔数	提取一层就停
文件 polyglot	一个文件能被两个程序同时识别	查头尾、对比不同解析器结果	以为“能打开”就说明只有一种格式
文档内嵌对象	PDF/Office 正常打开，但体积偏大	查对象、附件、元数据	只读正文，不查附件流

3. 密码学组合类

题型	入口特征	首选工具/动作	常见坑点
Base64 二层载荷	长文本只含 Base64 字符集	先解码，再判断是否压缩/加密	Base64 解出来乱码就放弃
OpenSSL `Salted__`	Base64 以 `U2FsdGVkX1` 开头	识别 `Salted__`、尝试 `AES-CBC`、找口令来源	没把假 flag、文件名当密码候选
压缩流再编码	`H4sIA`、`eJ`、十六进制流	先解码再解压	顺序搞反导致一直失败

题型	入口特征	首选工具/动作	常见坑点
假 flag 诱饵	已拿到像 flag 的东西，但题目过于简单	看同文件里是否还有高熵数据	见到 `flag{}` 就停
分层密码	提取结果仍像密文或口令提示	先确认算法头，再试来源明确的口令	一上来就无脑字典爆破

4. 音频类

题型	入口特征	首选工具/动作	常见坑点
频谱写字	肉耳不明显，但高频可能有形状	Audacity 频谱视图、调 FFT	只听不看
左右声道差分	两边听感不同或波形差异大	分离左右声道、做差分	只听混合声道
反放/变速	听起来奇怪、节奏不自然	反向、变速、截取局部	以为是噪声就跳过
音频 LSB	多见于 WAV	专项工具或脚本提取位数据	压缩音频上硬套未压缩方法
摩斯/DTMF	节奏明显、音调离散	摩斯码、电话按键音识别	把有规律节奏当背景音

5. 视频类

题型	入口特征	首选工具/动作	常见坑点
拆帧藏图	正常播放看不清，逐帧可见	`ffmpeg` 抽帧	只靠肉眼看视频播放
间隔帧线索	奇偶帧、每秒一帧、每 N 帧有内容	按频率采样抽帧	抽太密或太疏都可能漏
视频里藏音频题	画面正常但音轨可疑	抽音轨、看频谱	只看画面，不管音轨
字幕/附件流	容器里可能多条流	`ffprobe` 看 streams	以为 MP4 只有画面和声音
闪帧二维码	播放时一闪而过	降速播放、抽帧、拼关键帧	看到了异常但没保存关键帧

6. 网络与流量类

题型	入口特征	首选工具/动作	常见坑点
DNS 子域编码	请求很多、子域名长且像编码串	导出域名、解 Base32/Base64/Hex	只看 IP 不看查询名
HTTP Header 隐写	请求体正常，但头字段可疑	导出 headers，关注顺序/值/大小写	只盯响应正文
时间间隔编码	内容正常，但时间分布异常	看时间序列、统计间隔	忽略时序信息
包长度编码	包长有规律	导出长度序列、映射二进制	只看 payload

7. 赛时最短决策顺序

```
1. 先确认真实格式
2. 先查头尾、元数据、可打印字符串
3. 先提取追加数据和中间载荷
4. 再判断是不是 Base64/压缩/密文
5. 结构没异常，再上通道、LSB、频域、音视频专项
```

8. 最值得优先记住的 8 条

1. `JPEG` 先看 `FFD9` 后有没有尾巴
2. `PNG` 先看 `IEND` 后有没有尾巴和异常 chunk
3. 看到 `U2FsdGVkX1` 就想到 `Salted__`
4. 看到 `PK` 就先怀疑压缩包
5. `JPEG` 不要默认优先查 LSB
6. 音频题不要只听，要看频谱和声道
7. 视频题不要只看播放，要拆帧和拆音轨
8. 拿到中间结果后，先问“这是终点还是下一层”

Practice 实操练习包

这里不是纯文档，而是一组可以直接动手做的练习题。

目录

- `01-jpeg-tail-zip/`：JPEG 尾部拼接 ZIP，练“查文件尾 + 提取压缩包”
- `02-fake-flag-openssl/`：假 flag + OpenSSL `Salted__`，练“两层拆解”
- `03-png-alpha-hidden/`：PNG alpha 通道藏字，练“看透明层”
- `04-low-contrast-png/`：低对比度隐藏文字，练“拉对比度”
- `05-base64-gzip-chain/`：编码链练习，练“Base64 + Gzip”
- `06-audio-morse-basic/`：音频基础练习，练“听感 + 波形/频谱”
- `07-steghide-style-jpeg/`：模拟 steghide 做题路径的 JPG 口令题，练“找口令 + 提取注释载荷 + 解密”
- `08-png-palette-order/`：PNG 调色板顺序题，练“看 PLTE 而不是只看像素”
- `09-gif-frame-flash/`：GIF 拆帧题，练“逐帧查看闪现信息”
- `10-video-frame-sampling/`：视频抽帧题，练“按时间采样抓关键帧”
- `11-png-lsb-zsteg/`：PNG LSB 题，练“位平面/最低位/自定义提取”
- `12-jpeg-xmp-clue/`：JPEG XMP 元数据题，练“先查 metadata 再谈解密”
- `13-audio-spectrum-text/`：音频频谱写字题，练“看 spectrogram 而不是只听”
- `14-layered-metadata-cipher/`：元数据密码 + 密文载荷双层题
- `15-image-metadata-cipher-3layer/`：图片 + 元数据 + 密文 + 假 flag 三层综合题
- `16-gif-qr-base64/`：GIF 中闪现二维码，二维码里是 Base64
- `17-pcap-dns-hidden/`：流量包入门题，练“看 DNS 查询名”
- `writeups-index.md`：全套练习的统一答案索引表

使用方式

每题目录下都包含：

- `challenge.md`：题目说明
- `hint.md`：提示
- `solution.md`：参考解法
- `files/`：样本附件

建议顺序：

1. 先只看 `challenge.md`
2. 做不出来再看 `hint.md`
3. 最后再对照 `solution.md`

推荐顺序

1. `01-jpeg-tail-zip`
2. `02-fake-flag-openssl`
3. `03-png-alpha-hidden`

4. `04-low-contrast-png`
5. `05-base64-gzip-chain`
6. `06-audio-morse-basic`
7. `07-steghide-style-jpeg`
8. `08-png-palette-order`
9. `09-gif-frame-flash`
10. `10-video-frame-sampling`
11. `11-png-lsb-zsteg`
12. `12-jpeg-xmp-clue`
13. `13-audio-spectrum-text`
14. `14-layered-metadata-cipher`
15. `15-image-metadata-cipher-3layer`
16. `16-gif-qr-base64`
17. `17-pcap-dns-hidden`

说明

- 样本文件由教程附带脚本生成，方便你反复重建
- 题目刻意控制在入门到初中级难度，重点是练流程
- 这套练习不追求刁钻，而是追求“每一题都能巩固一类方法”
- 部分媒体题和综合题的重建依赖 `scripts/media_builder/` 下的 Node 依赖；练习附件本身已经随教程一并提供

Practice Writeups Index

这是一份统一答案索引，方便批量复盘时对照。

基础批次

题号	目录	答案
01	`01-jpeg-tail-zip`	`flag{tail_zip_found}`
02	`02-fake-flag-openssl`	`flag{openssl_layer_clear}`
03	`03-png-alpha-hidden`	`flag{alpha_layer_reveal}`

题号	目录	答案
04	`04-low-contrast-png`	`flag{contrast_is_key}`
05	`05-base64-gzip-chain`	`flag{decode_then_decompress}`
06	`06-audio-morse-basic`	`AUDIO_MORSE_06`

进阶批次

题号	目录	答案
07	`07-steghide-style-jpeg`	`flag{comment_payload_opened}`
08	`08-png-palette-order`	`flag{palette_order_matters}`
09	`09-gif-frame-flash`	`flag{gif_frames_tell_all}`
10	`10-video-frame-sampling`	`flag{sample_video_frames}`
11	`11-png-lsb-zsteg`	`flag{lsb_bits_signal}`
12	`12-jpeg-xmp-clue`	`flag{xmp_reveals_truth}`
13	`13-audio-spectrum-text`	`FLAG{SPECTRUM_13}`
14	`14-layered-metadata-cipher`	`flag{metadata_unlocks_cipher}`

综合批次

题号	目录	答案
15	`15-image-metadata-cipher-3layer`	`flag{triple_layers_pay_off}`
16	`16-gif-qr-base64`	`flag{qr_frames_base64}`
17	`17-pcap-dns-hidden`	`flag{dns_queries_whisper}`